

Worked for 1m 11s

I inspected the whole project and the README. There are **4 required tasks**, and the cleanest solution is to replace **2 files completely**: `game/game_engine.py` and `main.py`. `bird.py`, `pipe.py`, and `requirements.txt` can stay as they are.

I also checked the replacement code for Python syntax successfully. I couldn't run the actual Pygame window in this environment because Pygame isn't installed here, but the code is written against the existing project structure and `requirements.txt`.

1. What is wrong in your original code?

Task 1 — Collision Detection

Your original code has:

Python

Run

```
if pipe.top_rect().collidepoint(self.bird.center()) or pipe.bottom_rect().collidepoint(se
```

The problem is that this checks **only one point**: the center of the bird.

Your bird is actually a circle with a radius of 15.

So imagine:

PIPE



The bird can visually touch the pipe with its outer edge while its **center is still outside the pipe**.

Therefore:

Python

Run

```
collidepoint(self.bird.center())
```

can return `False`.

That's exactly the bug described in the README.

Fix

We use:

Python

Run

```
bird_rect = self.bird.rect()
```

and then:

Python

Run

```
bird_rect.collidect(top_rect)
```

This checks the **whole bird area**, not just its center.

I also slightly enlarge the pipe collision rectangle:

Python

Run

```
collision_margin = 2
top_rect = pipe.top_rect().inflate(
    collision_margin * 2,
    collision_margin * 2
)
```

This makes edge collisions reliable.

2. Task 2 — Game Over Screen

Originally, when the player died, your code eventually did:

Python

Run

```
print("Game over! Final score:", self.score)
```

That only prints something in the terminal.

The README specifically says:

Add a screen that displays the final score...

So we replace that with an actual Pygame overlay:

GAME OVER

Final Score: 3

E: Easy M: Medium H: Hard

The game also stops updating while this screen is displayed.

3. Task 3 — Replay + Difficulty

Originally you had:

Python

Run

```
self.pipe_speed = 4
self.pipe_interval = 90
```

and they never changed.

Now we have:

Python

Run

```
DIFFICULTIES = {
    "Easy": {"speed": 3, "gap": 180, "interval": 100},
    "Medium": {"speed": 4, "gap": 150, "interval": 90},
    "Hard": {"speed": 6, "gap": 125, "interval": 80},
}
```

So:

Difficulty	Pipe speed	Gap	Spawn interval
Easy	3	180	100
Medium	4	150	90
Hard	6	125	80

Therefore Hard is actually harder in **three ways**:

- Pipes move faster.
- Gap is smaller.
- Pipes appear more frequently.

After Game Over:

E → Easy

M → Medium

H → Hard

Q / ESC → Exit

4. Task 4 — Sound

Your original project has no sound at all.

I deliberately did **not** make you download `.wav` files and put them somewhere, because that creates another opportunity for path/file errors.

Instead, the game creates tiny sound effects programmatically.

There are three:

- 🦋 Flap sound
- ★ Score/pass-pipe sound
- 💀 Death sound

So you don't need to add any additional files.

EXACT CHANGES TO MAKE

File 1: `game/game_engine.py`

Replace the ENTIRE file

Don't try to modify individual lines here.

Open:

```
40_flappy-bird-main
├── game
│   └── game_engine.py
```

Delete everything currently inside `game_engine.py`.

Then paste this **entire code**:

Python

Run

```
import math
from array import array

import pygame
```



```

        0.12,
        0.25
    ),

    "death": self._create_tone(
        [250, 100],
        0.25,
        0.30
    ),
}

def _play_sound(self, name):

    sound = self.sounds.get(name)

    if sound is not None:
        sound.play()

def reset(self, difficulty="Medium"):
    """
    Start a fresh game using the selected difficulty.
    """

    if difficulty not in self.DIFFICULTIES:
        difficulty = "Medium"

    settings = self.DIFFICULTIES[difficulty]

    self.difficulty = difficulty

    self.pipe_speed = settings["speed"]
    self.pipe_gap = settings["gap"]
    self.pipe_interval = settings["interval"]

    self.bird = Bird(
        self.width // 4,
        self.height // 2
    )

    self._spawn_timer = 0

    self.pipes = [
        Pipe(
            self.width + 100,
            self.height,
            gap=self.pipe_gap,
            speed=self.pipe_speed
        )
    ]

    self.score = 0

```



```

        self._play_sound("flap")

# Mouse click
elif event.type == pygame.MOUSEBUTTONDOWN:

    self.bird.flap()

    # Play flap sound
    self._play_sound("flap")

    return None

def handle_input(self):
    """
    No continuously-held input is needed.
    """

    pass

def _set_game_over(self):

    if not self.game_over:

        self.game_over = True

        # Play death sound only once
        if not self._game_over_sound_played:

            self._play_sound("death")

            self._game_over_sound_played = True

def update(self):

    # Do not update anything after Game Over
    if self.game_over:
        return

    # -----
    # UPDATE BIRD
    # -----

    self.bird.update()

    # -----
    # CHECK GROUND / CEILING
    # -----

    if (
        self.bird.y - self.bird.radius <= 0
        or
        self.bird.y + self.bird.radius >= self.height

```

```

):

    self._set_game_over()

    return

# -----
# SPAWN PIPES
# -----

self._spawn_timer += 1

if self._spawn_timer >= self.pipe_interval:

    self._spawn_timer = 0

    self.pipes.append(
        Pipe(
            self.width,
            self.height,
            gap=self.pipe_gap,
            speed=self.pipe_speed
        )
    )

# -----
# COLLISION DETECTION
# -----

# IMPORTANT:
# Use the complete bird rectangle instead
# of checking only the bird's center point.

bird_rect = self.bird.rect()

for pipe in self.pipes:

    # Move pipe
    pipe.move()

    # -----
    # TASK 1 FIX
    # -----

    # Small collision margin makes edge hits
    # more reliable.

    collision_margin = 2

    top_rect = pipe.top_rect().inflate(
        collision_margin * 2,
        collision_margin * 2
    )

```



```

screen.blit(
    difficulty_text,
    (10, 45)
)

# -----
# TASK 2 FIX:
# GAME OVER SCREEN
# -----

if self.game_over:

    # Transparent black overlay
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )

    overlay.fill(
        (0, 0, 0, 150)
    )

    screen.blit(
        overlay,
        (0, 0)
    )

    # GAME OVER
    title = self.big_font.render(
        "GAME OVER",
        True,
        WHITE
    )

    # FINAL SCORE
    final_score = self.font.render(
        f"Final Score: {self.score}",
        True,
        WHITE
    )

    # DIFFICULTY OPTIONS
    choose = self.small_font.render(
        "E: Easy   M: Medium   H: Hard",
        True,
        WHITE
    )

    # EXIT
    exit_text = self.small_font.render(
        "Q or ESC: Exit",

```

```

        True,
        WHITE
    )

    # Position everything

    screen.blit(
        title,
        title.get_rect(
            center=(
                self.width // 2,
                240
            )
        )
    )

    screen.blit(
        final_score,
        final_score.get_rect(
            center=(
                self.width // 2,
                310
            )
        )
    )

    screen.blit(
        choose,
        choose.get_rect(
            center=(
                self.width // 2,
                380
            )
        )
    )

    screen.blit(
        exit_text,
        exit_text.get_rect(
            center=(
                self.width // 2,

```

5. File 2: `main.py`

Now open:

```

40_flappy-bird-main
└─ main.py

```

Again, **delete the entire existing file.**

Your original `main.py` is only about 42 lines, but it doesn't know how to restart the game after Game Over.

Replace the entire thing with:

```
Python
import pygame

from game.game_engine import GameEngine

# -----
# INITIALIZE PYGAME AUDIO
# -----

pygame.mixer.pre_init(
    frequency=44100,
    size=-16,
    channels=1
)

pygame.init()

# -----
# SCREEN
# -----

WIDTH, HEIGHT = 500, 700

SCREEN = pygame.display.set_mode(
    (WIDTH, HEIGHT)
)

pygame.display.set_caption(
    "Flappy Bird - Pygame Version"
)

# -----
# COLORS
# -----

SKY_BLUE = (135, 206, 235)

# -----
# CLOCK
```

Run


```

        if action == "quit":

            running = False

            continue

        # -----
        # PLAYER SELECTED A DIFFICULTY
        # -----

        if action in GameEngine.DIFFICULTIES:

            # Completely restart the game
            # with the selected difficulty.

            engine.reset(action)

        # -----
        # UPDATE GAME
        # -----

        if running:

            engine.handle_input()

            engine.update()

            engine.render(SCREEN)

            pygame.display.flip()

            clock.tick(FPS)

        # -----
        # CLEANUP
        # -----

        pygame.quit()

    # -----
    # START PROGRAM
    # -----

if __name__ == "__main__":

    main()

```

6. Do NOT change these files

You can leave these exactly as they currently are:

game/bird.py

No change needed.

Your existing:

Python

Run

```
def rect(self):
    return pygame.Rect(
        self.x - self.radius,
        self.y - self.radius,
        self.radius * 2,
        self.radius * 2
    )
```

is actually useful for our collision fix.

game/pipe.py

No change needed.

The existing:

Python

Run

```
def top_rect(self):
    return pygame.Rect(
        self.x,
        0,
        self.width,
        self.gap_y
    )
```

and:

Python

Run

```
def bottom_rect(self):
    bottom_y = self.gap_y + self.gap

    return pygame.Rect(
        self.x,
        bottom_y,
```

```
self.width,  
self.screen_height - bottom_y  
)
```

are correct.

```
requirements.txt
```

Keep:

```
pygame
```

No additional library is required for the new features.

7. How each README requirement is now satisfied

This is important because your teacher may ask you this during the demonstration.

Task 1 — Refine Collision Detection

Before

You had:

Python

Run

```
pipe.top_rect().collidepoint(self.bird.center())
```

Meaning:

"Is the CENTER of the bird inside the pipe?"

After

We have:

Python

Run

```
bird_rect = self.bird.rect()
```

and:

Python

Run

```
bird_rect.colliderect(top_rect)
```

Meaning:

"Does the bird's entire rectangular area overlap the pipe?"

And:

Python

Run

```
collision_margin = 2
```

adds a small safety margin for edge contact.

What you can tell your teacher

"The original implementation checked only the bird's center point using `collidepoint()`. This caused edge clipping because the bird's body could overlap the pipe while its center remained outside. I changed it to rectangle-based collision using `colliderect()` and added a small collision margin for more reliable edge detection."

That's a **very good answer** for Task 1.

8. Task 2 — Game Over

Before

You had:

Python

Run

```
print("Game over! Final score:", self.score)
```

So the game only told you through the terminal.

After

We have:

Python

Run

```
if self.game_over:
```

and draw:

Python

Run

```
"GAME OVER"
```

plus:

Python

Run

```
f"Final Score: {self.score}"
```

What you can tell your teacher

"Previously Game Over was only printed to the console. I added an in-game overlay that displays GAME OVER and the final score. Once `game_over` becomes true, the update function returns immediately, so the game stops and waits for the player's input."

9. Task 3 — Replay + Difficulty

The important part is:

Python

Run

```
DIFFICULTIES = {  
    "Easy": {"speed": 3, "gap": 180, "interval": 100},  
    "Medium": {"speed": 4, "gap": 150, "interval": 90},  
    "Hard": {"speed": 6, "gap": 125, "interval": 80},  
}
```

Then when Game Over happens:

```
E → Easy  
M → Medium  
H → Hard  
Q → Exit
```

For example, pressing `H` returns:

Python

Run

```
return "Hard"
```

Then `main.py` receives it:

Python

Run

```
if action in GameEngine.DIFFICULTIES:  
    engine.reset(action)
```

and the game starts again.

What you can tell your teacher

"I implemented three difficulty levels by changing pipe speed, gap size and spawn interval. After Game Over, the player can choose Easy, Medium or Hard, and the

`reset()` method creates a fresh game with those settings."

10. Task 4 — Sound

There are three calls.

Flap

When Space or mouse is pressed:

Python

Run

```
self._play_sound("flap")
```

Score

When a pipe is successfully passed:

Python

Run

```
self.score += 1  
self._play_sound("score")
```

Death

When collision happens:

Python

Run

```
self._set_game_over()
```

and inside that:

Python

Run

```
self._play_sound("death")
```

The death sound is protected by:

Python

Run

```
self._game_over_sound_played
```

so it doesn't repeatedly play every frame.

What you can tell your teacher

"I added three sound effects: flap, score and death. I generated them in memory instead of depending on external audio files, so the project doesn't require additional assets. The death sound is triggered only once when the game transitions into the Game Over state."

11. Your final folder should look like this

```
40_flappy-bird-main/
|
├─ main.py           ← REPLACED
|
├─ requirements.txt  ← NO CHANGE
|
├─ README.md         ← NO CHANGE
|
├─ game/
|   |
|   ├─ bird.py       ← NO CHANGE
|   |
|   ├─ pipe.py       ← NO CHANGE
|   |
|   └─ game_engine.py ← REPLACED
```

You do not need to create any sound files.

12. How to run it

Open the terminal inside:

```
40_flappy-bird-main
```

Then:

Bash

```
pip install -r requirements.txt
```

Then:

Bash

```
python main.py
```

13. What you should demonstrate in your video

The README asks for a **10-second video before** and a **10-second video after**.

For the **before video**, use your original code and show the broken behavior.

For the **after video**, I would demonstrate it in this order:

① Start the game

Show the bird falling.

② Press Space

Show:



and hear the flap sound.

③ Pass a pipe

Show:

```
Score: 1
```

and hear the scoring sound.

④ Hit a pipe

Now the screen should become:

```
GAME OVER
```

```
Final Score: 1
```

```
E: Easy   M: Medium   H: Hard
```

```
Q or ESC: Exit
```

⑤ Press H

The game should restart.

Now the pipes should be:

- faster
- narrower gaps
- more frequent

⑥ Optionally die again and press E

This demonstrates that the replay/difficulty selection actually works.

That gives you a very clean demonstration of **all four README tasks**.

One important thing

Don't just say **"I changed the collision code."** If your teacher asks what the actual bug was, the key distinction is:

```
OLD:
collidepoint(bird.center())
    ↓
checks ONE POINT

NEW:
bird.rect().colliderect(pipe_rect)
    ↓
checks the BIRD'S AREA
```

That is the core technical fix.

And for the replay:

```
Game Over
    ↓
E / M / H
    ↓
engine.reset(difficulty)
    ↓
new bird + new pipes + score = 0
```

So you can explain the whole implementation rather than simply saying "ChatGPT fixed it."

Do you like this personality?

☐